AIGC 5005 | MODULE 4 | 3.5% OF FINAL GRADE

Lab 04: Harden Your Package

Due date posted on Blackboard

Take the package you built for Lab 03 and make it trustworthy. You will add an automated test suite, errors that explain themselves, logging in place of print statements, type hints that a checker accepts, formatting and linting that come back clean, and a settings file and seed that let anyone repeat a run. The Module 4 worked example did this to a library catalogue. This lab asks you to do it to your own code.

The package should still do what it did before. The same data goes in and the same summary comes out, with one addition: a short list of featured shows chosen at random from a seed that you set. What changes is how far you can rely on the program, and how quickly you would find out if something broke.

What this lab is about

Lab 03 was marked on design. Lab 04 is marked on reliability. If someone changed your code tomorrow, would a test tell them what they broke? If the network failed, would the message help? If a marker ran your program twice, would they get the same answer? A package that can answer yes to all three is one that other people, and you in three months, can depend on.

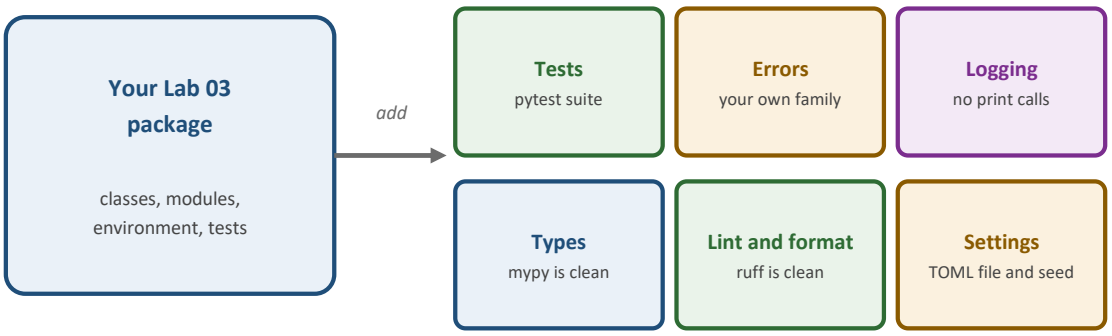


Figure 1. Lab 04 keeps your Lab 03 design and adds six habits around it.

You start from your own Lab 03 repository. There is nothing new to download and no new data to learn. All of the effort goes into making the code you already have dependable.

What your package must contain

Each of the following is checked directly during marking.

1. A test suite. At least twelve tests in a tests folder that pass with python -m pytest and need no network. The suite must include: tests of your record class that cover a missing genre list and a missing rating; a test for each aggregation class using hand-made records; at least one parametrized test; at least one test that uses pytest.raises with a match argument; at least one fixture in conftest.py; one test that uses tmp_path; and tests of main() that check the exit code and the printed message for a failed run.
2. Errors that explain themselves. A base exception class for your package and at least three subclasses for failures the program can expect, such as the download failing, the response not having the shape you expect, and the settings file being unusable. Translate low-level errors with raise ... from error. No bare except, and no except Exception that hides a failure. Only main() catches your package's errors.

3. Logging instead of printing. Every module that has something to report creates logger = logging.getLogger(__name__). No print calls anywhere in the package except the final user-facing output in main(). Configure logging once, in main(), using a function in logging_setup.py. Your code must log at DEBUG (for example, how many records arrived), INFO (milestones), WARNING (a record skipped because of bad data) and ERROR (the failure that stopped the run). Pass values as arguments in the %s style, not through f-strings, and never log secrets.
4. A real entry point. A function main(argv=None) -> int that lives in the package as main.py and runs with python -m yourpackage.main. It accepts a --config option. It returns 0 on success. On an expected failure it logs the error, prints one line beginning Error: to standard error, and returns 1, with no traceback. If your Lab 03 main.py sits outside the package, move it inside. The summary keeps every key and value it had in Lab 03.
5. Type hints that pass. Annotate every function and method in the package, including return types, using built-in generics such as list[str] and str | None. python -m mypy reports no issues with disallow_untyped_defs switched on in pyproject.toml. Include an empty py.typed file in the package. Use at most two type: ignore comments, each with the reason written beside it.
6. Formatting and linting that pass. python -m ruff format --check . and python -m ruff check . both pass, with the rule families E, F, I, B and UP selected in pyproject.toml. Any noqa comment has a reason beside it.
7. Settings and reproducibility. Replace the constants in your config module with a TOML file at config/yourpackage.toml, read with tomllib into a frozen dataclass. It holds the source URL, the timeout, the output folder, a seed, a sample size and the log level. A missing or incomplete file raises your settings error. Add a featured_shows list to the summary, containing sample_size show names chosen with random.Random(seed). The same seed must give the same list, and a test must prove it. Each run also writes run_info.json next to summary.json, recording the settings used, the Python version, and the versions of pytest, mypy and ruff.
8. Environment and README. An environment.yml that fixes the Python version, a requirements.txt with pinned versions that now includes pytest, mypy and ruff, and a README that follows the template below, including the Check it and Errors and exit codes sections.

Required layout

Keep the names from your Lab 03 package. The new files and folders are marked in the table below. The shape is not negotiable, although the names inside it are suggestions.

```
your-repository/
    README.md
    environment.yml          conda environment: name, Python version, pip
    requirements.txt         pinned pip packages, including pytest, mypy, ruff
    pyproject.toml           package, pytest, ruff and mypy settings
    .gitignore
    config/
        yourpackage.toml     NEW  every setting, including the seed
    src/
        yourpackage/
            __init__.py      re-exports the public names
            py.typed         NEW  empty marker: this package has type hints
            errors.py        NEW  the base exception and its subclasses
            config.py        reads the TOML file into a frozen Settings object
            logging_setup.py NEW  configure_logging(level)
            models.py        the record class
            sources.py       fetching; only module using requests
            aggregations.py  base class and subclasses
```

```
        report.py        build and write summary and run_info
        main.py          entry point: python -m yourpackage.main
tests/
    conftest.py          NEW  shared fixtures
    test_models.py
    test_aggregations.py
    test_sources.py
    test_main.py
data/
    raw/.gitkeep
    processed/summary.json  run_info.json
```

You may add more modules if your design calls for them, for example a separate module for run_info.json. You may not move code back into one big module to make the checks easier to pass.

Suggested order of work

This is guidance, not a requirement, but it keeps a large change from turning into a large mess. Commit after each step.

1. Create a new repository. Your first commit is your Lab 03 package, unchanged, so the history shows what you started from. Create a fresh conda environment, install pytest, mypy and ruff with pip, and record the versions in requirements.txt.
2. Run ruff format and ruff check --fix on the existing code, and commit the result on its own. Mixing tidy-up changes with behaviour changes makes both hard to review.
3. Write tests for the code as it is today, starting with the record class. Get them passing before you change any behaviour.
4. Add errors.py and use it. Translate low-level errors at the boundary, and add tests with pytest.raises for each one.
5. Add logging. Replace each print call, add logging_setup.py, and test one logged message with caplog.
6. Add type hints and the mypy settings, then fix what mypy reports.
7. Move the settings into the TOML file. Add the seed, featured_shows and run_info.json, and test that the same seed gives the same list.
8. Move the entry point into the package and add the exit codes. Run all four checks, run the program, and update the README.

> **Check that nothing changed**
>
> Keep your Lab 03 summary.json. After hardening, compare it with the new one, for example in VS Code by right-clicking one file and choosing Select for Compare. Every key and value from Lab 03 should be identical, and the only new key should be featured_shows.
>
> A failing check after a change is useful information: either your change altered behaviour, or the old behaviour was a bug. Decide which, and say so in the commit message.

Testing without the network

Your tests must not depend on the internet, because a test that fails when the wifi does is a test nobody trusts. Most of your suite will use hand-made records. The one place that needs more care is the failure path

in your source module. The monkeypatch fixture replaces a function for the duration of one test, so you can make requests.get fail on demand. The names below are examples; use your own.

```
import pytest
import requests

from yourpackage import sources
from yourpackage.errors import SourceError


def test_a_failed_download_raises_a_source_error(monkeypatch):
    def fake_get(*args, **kwargs):
        raise requests.ConnectionError("no network")

    monkeypatch.setattr(sources.requests, "get", fake_get)
    with pytest.raises(SourceError, match="reach"):
        sources.fetch_shows("https://example.invalid", timeout=1)
```

This also tells you something about your design. If your fetching function is hard to test, it is probably doing too much. Splitting it into one function that downloads and one that turns the downloaded list into records makes the second one trivial to test.

Check your work with four commands

These four commands are the standard check from Module 4. All four must pass before you commit a change, and all four are run on a fresh clone when your lab is marked.

```
python -m ruff format --check .
python -m ruff check .
python -m mypy
python -m pytest
```

Settings for all three tools live in pyproject.toml, so that you and the marker get identical results. Add these sections to the file you already have.

```
[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
src = ["src", "tests"]

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP"]

[tool.mypy]
python_version = "3.11"
files = ["src"]
disallow_untyped_defs = true
warn_return_any = true
warn_unused_ignores = true
```

If mypy complains that the requests library has no type information, install the stubs with pip install types-requests and add them to requirements.txt.

Environment files

The rules are the same as in Lab 03, with two changes: the environment is new, because this is a new repository, and it now includes the three checking tools. Use the conda-forge channel for the Python install.

environment.yml

```
name: yourpackage
channels:
  - conda-forge
dependencies:
  - python=3.11
  - pip
```

requirements.txt

```
conda env create -f environment.yml
conda activate yourpackage
pip install requests pytest mypy ruff
pip freeze > requirements.txt
pip install -e .
```

> **Watch out**
>
> Run pip freeze before pip install -e . If you freeze after the editable install, requirements.txt gains a line starting with -e and an absolute path from your own computer, which will fail on every other machine. If you already have that line, delete it.
>
> Do not submit the output of conda env export. It lists every package for your operating system and will not rebuild on a classmate's machine.

Repository requirements

Create a new repository for this lab. Do not add the work to your Lab 01, Lab 02 or Lab 03 repository.

Item	Requirement
.gitignore	Committed before your first code commit, excluding __pycache__/, *.egg-info/, .env, .pytest_cache/, .mypy_cache/ and .ruff_cache/. Do not ignore data/processed/ in this lab; the marker needs to see your output
README.md	Follows the template below, including Check it, Errors and exit codes, and Reproducing a run
environment.yml	Conda environment as described above, using conda-forge
requirements.txt	Pinned versions that include pytest, mypy and ruff, with no absolute paths

Item	Requirement
pyproject.toml	Package name, version and dependencies, with src layout, plus the pytest, ruff and mypy sections
config/	The TOML settings file, with every setting your program reads
src/	The package, including main.py, errors.py, logging_setup.py and py.typed
tests/	At least twelve passing tests that need no network, with fixtures in conftest.py
data/processed/	summary.json and run_info.json from a successful run
Commit history	At least eight commits, each a coherent change with a message that says what changed. The first is your Lab 03 package, unchanged. Formatting-only changes are in their own commit

README template

Replace the content, not the shape. The Errors and exit codes and Reproducing a run sections carry marks in this lab.

```
# Project Name

One or two sentences on what this package summarizes.

## Setup
conda env create -f environment.yml
conda activate yourpackage
pip install -r requirements.txt
pip install -e .

## Run
python -m yourpackage.main
python -m yourpackage.main --config config/yourpackage.toml

## Check it
python -m ruff format --check .
python -m ruff check .
python -m mypy
python -m pytest

## Settings
One line for each setting in the TOML file: what it controls.

## Errors and exit codes
A table with three columns: each error class, when it is raised,
and what the user sees. State what exit code 0 and 1 mean.

## Reproducing a run
Which file holds the seed, what run_info.json records, and what else
a classmate would need to get exactly the same featured_shows list.

## What the tests cover
The behaviour your tests protect, and one thing you chose not to test
and why.
```

```
## Known limitations
Anything that does not work, or that you would improve with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 04 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

> **Instructor GitHub account**
>
> hossein.pourmodheji@humber.ca
>
> Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Late work

Late submissions are accepted with a deduction of 5% per day, to a maximum of five days. After five days the submission receives a grade of zero.

> **Watch out**
>
> Before you submit, clone your repository into a different folder. Create the environment from environment.yml, install requirements.txt and the package, then run the four checks and the program. Finally, run the program with --config pointing at a file that does not exist, and confirm that you get one line beginning Error: and an exit code of 1. A repository that only works in your original folder has not passed.

> **Looking ahead to Module 05**
>
> Module 05 moves to machine learning with scikit-learn. Everything in this lab carries over: a model that is trained with a recorded seed and settings file, and covered by a few tests, is one you can compare fairly and trust. Keep this repository tidy, because its structure and checks are what you will start from.